트랜잭션 처리 플로우 분석

1. 트랜잭션 처리 개요

MiniEVM에서 트랜잭션 처리는 Ethereum 호환성과 Cosmos SDK 통합을 동시에 제공하는 복잡한 과정입니다.

1.1 트랜잭션 타입

- **EVM 트랜잭션**: Ethereum 형식의 트랜잭션 (Legacy, EIP-1559)
- **Cosmos 트랜잭션**: Cosmos SDK 네이티브 트랜잭션
- **하이브리드 트랜잭션**: EVM 트랜잭션을 Cosmos 트랜잭션으로 래핑

2. 전체 트랜잭션 플로우

```
Client Request

↓

JSON-RPC Server (eth_sendRawTransaction)

↓

Transaction Parsing & Validation

↓

Cosmos Transaction Wrapping

↓

Ante Handler Processing
```

```
↓
Mempool Addition
↓
Consensus (CometBFT)
↓
Block Proposal & Validation
↓
EVM Execution
↓
Post Handler Processing
↓
State Commitment
↓
Indexer Processing
↓
Event Emission
```

3. JSON-RPC 트랜잭션 수신

3.1 eth_sendRawTransaction 처리

```go
func (api *PublicAPI) SendRawTransaction(ctx context.Context, input hexutil.Byte

// 1. 트랜잭션 파싱
```

```go
tx := new(types.Transaction)
if err := tx.UnmarshalBinary(input); err != nil {
return common.Hash{}, err
}
// 2. 백엔드로 전달
return api.backend.SendRawTransaction(ctx, tx)
}
```

3.2 트랜잭션 검증

```go
func (b *Backend) SendRawTransaction(ctx context.Context, tx *types.Transaction)
// 1. 기본 검증
if tx.To() == nil && len(tx.Data()) == 0 {
return common.Hash{}, errors.New("contract creation without data")
}
// 2. 서명 검증
signer := types.LatestSignerForChainID(b.chainID)
from, err := types.Sender(signer, tx)
if err != nil {
return common.Hash{}, err
}
// 3. 논스 검증
```

```
nonce := b.GetNonce(from, rpc.LatestBlockNumber)

if  tx.Nonce() < nonce {

return common.Hash{}, errors.New("nonce too low")

}

return  tx.Hash(), b.sendTransaction(ctx, tx)

}
```

4. Cosmos 트랜잭션 변환

4.1 EVM → Cosmos 트랜잭션 래핑

```
func (b *Backend) sendTransaction(ctx context.Context, tx *types.Transaction) er

// 1. EVM 트랜잭션을 Cosmos 메시지로 변환

msg := &evmtypes.MsgEthereumTx{

Data: &evmtypes.LegacyTx{

Nonce: tx.Nonce(),

GasPrice: tx.GasPrice(),

Gas: tx.Gas(),

To: tx.To().Hex(),

Value: tx.Value(),

Data: tx.Data(),

V: tx.V(),

R: tx.R(),
```

```
S: tx.S(),
},
}
// 2. Cosmos 트랜잭션 생성
txBuilder := b.clientCtx.TxConfig.NewTxBuilder()
if err := txBuilder.SetMsgs(msg); err != nil {
return err
}
// 3. 트랜잭션 인코딩
txBytes, err := b.clientCtx.TxConfig.TxEncoder()(txBuilder.GetTx())
if err != nil {
return err
}
// 4. 메모리풀에 추가
return b.app.CheckTx(&abci.RequestCheckTx{
Tx: txBytes,
Type: abci.CheckTxType_New,
})
}
```

5. CheckTx 처리

5.1 CheckTx 핸들러

```go
func (app *MinitiaApp) CheckTx(req *abci.RequestCheckTx) (*abci.ResponseCheckTx,
return app.checkTxHandler(req)
}
```

5.2 BlockSDK CheckTx 처리

```go
func (h *CheckTxHandler) CheckTx(req *abci.RequestCheckTx) (*abci.ResponseCheckT
// 1. 트랜잭션 디코딩
tx, err := h.txDecoder(req.Tx)
if err != nil {
return &abci.ResponseCheckTx{Code: 1, Log: err.Error()}, nil
}
// 2. Ante Handler 실행
ctx := h.app.NewContext(true, tmproto.Header{})
_, err = h.anteHandler(ctx, tx, false)
if err != nil {
return &abci.ResponseCheckTx{Code: 1, Log: err.Error()}, nil
}
// 3. 메모리풀 검증
if err := h.mempool.Insert(ctx, tx); err != nil {
return &abci.ResponseCheckTx{Code: 1, Log: err.Error()}, nil
}
```

```
return &abci.ResponseCheckTx{Code: 0}, nil

}
```

6. Ante Handler 처리

6.1 Ante Handler 체인

```
func  NewAnteHandler(options HandlerOptions) sdk.AnteHandler {

return  sdk.ChainAnteDecorators(

ante.NewSetUpContextDecorator(),

ante.NewExtensionOptionsDecorator(options.ExtensionOptionChecker),

ante.NewValidateBasicDecorator(),

ante.NewTxTimeoutHeightDecorator(),

ante.NewValidateMemoDecorator(options.AccountKeeper),

ante.NewConsumeGasForTxSizeDecorator(options.AccountKeeper),

ante.NewDeductFeeDecorator(options.AccountKeeper, options.BankKeeper, options.Fe

ante.NewSetPubKeyDecorator(options.AccountKeeper),

ante.NewValidateSigCountDecorator(options.AccountKeeper),

ante.NewSigGasConsumeDecorator(options.AccountKeeper, options.SigGasConsumer),

ante.NewSigVerificationDecorator(options.AccountKeeper, options.SignModeHandler)

ante.NewIncrementSequenceDecorator(options.AccountKeeper),

// EVM 특화 데코레이터들

NewEthAccountVerificationDecorator(options.AccountKeeper),
```

```
NewEthGasConsumeDecorator(options.EVMKeeper),
NewCanTransferDecorator(options.EVMKeeper),
NewEthIncrementSenderSequenceDecorator(options.AccountKeeper),
)
}
```

6.2 EVM 특화 Ante 데코레이터

EthAccountVerificationDecorator

```
func (avd EthAccountVerificationDecorator) AnteHandle(ctx sdk.Context, tx sdk.Tx
for _, msg := range tx.GetMsgs() {
msgEthTx, ok := msg.(*evmtypes.MsgEthereumTx)
if !ok {
continue
}
// 1. 발신자 주소 추출
from, err := msgEthTx.GetSender(ctx.ChainID())
if err != nil {
return ctx, err
}
// 2. 계정 존재 확인 및 생성
acc := avd.ak.GetAccount(ctx, from.Bytes())
if acc == nil {
```

```
acc = avd.ak.NewAccountWithAddress(ctx, from.Bytes())
avd.ak.SetAccount(ctx, acc)
}
}
return next(ctx, tx, simulate)
}
```

EthGasConsumeDecorator

```
func (egcd EthGasConsumeDecorator) AnteHandle(ctx sdk.Context, tx sdk.Tx, simula
for _, msg := range tx.GetMsgs() {
msgEthTx, ok := msg.(*evmtypes.MsgEthereumTx)
if !ok {
continue
}
// 1. 가스 가격 검증
gasPrice := msgEthTx.GetGasPrice()
minGasPrice := egcd.evmKeeper.GetMinGasPrice(ctx)
if gasPrice.Cmp(minGasPrice) < 0 {
return ctx, errors.New("gas price too low")
}
// 2. 가스 한도 검증
gasLimit := msgEthTx.GetGas()
```

```
if  gasLimit > egcd.evmKeeper.GetMaxGasLimit(ctx) {
return  ctx, errors.New("gas limit too high")
}
// 3. 가스 수수료 차감
gasFeeCoin := sdk.NewCoin(egcd.evmKeeper.GetEvmDenom(ctx), sdk.NewIntFromBigInt(
if  err := egcd.bankKeeper.SendCoinsFromAccountToModule(ctx, from, authtypes.Fee
return  ctx, err
}
}
return  next(ctx, tx, simulate)
}
```

7. 메모리풀 관리

7.1 메모리풀 구조

```
type Mempool interface {
// 트랜잭션 삽입
Insert(ctx context.Context, tx sdk.Tx) error
// 트랜잭션 선택 (블록 제안용)
Select(ctx context.Context, txs [][]byte) mempool.Iterator
// 트랜잭션 제거
Remove(tx sdk.Tx) error
```

```
// 메모리풀 상태
CountTx() int
}
```

7.2 우선순위 기반 메모리풀

```
type PriorityMempool struct {
// 우선순위 큐
txQueue *PriorityQueue
// 발신자별 트랜잭션 추적
senderTxs  map[string][]*MempoolTx
// 논스 추적
senderNonces  map[string]uint64
}

func (m *PriorityMempool) Insert(ctx context.Context, tx sdk.Tx) error {
// 1. 트랜잭션 우선순위 계산
priority := m.calculatePriority(tx)
// 2. 발신자별 논스 검증
sender := m.getSender(tx)
expectedNonce := m.senderNonces[sender]
if  tx.GetNonce() != expectedNonce {
return  errors.New("invalid nonce")
```

```
}

// 3. 우선순위 큐에 삽입

mempoolTx := &MempoolTx{

Tx: tx,

Priority: priority,

Sender: sender,

}

m.txQueue.Push(mempoolTx)

m.senderTxs[sender] = append(m.senderTxs[sender], mempoolTx)

m.senderNonces[sender]++

return nil

}
```

8. 블록 제안 및 검증

8.1 PrepareProposal

```
func (h *ProposalHandler) PrepareProposal(ctx sdk.Context, req *abci.RequestPrep

// 1. 메모리풀에서 트랜잭션 선택

iterator := h.mempool.Select(ctx, nil)

var txs [][]byte

totalSize := 0

for iterator.Valid() {
```

```go
tx := iterator.Tx()
txBytes, err := h.txEncoder(tx)
if err != nil {
iterator.Next()
continue
}
// 2. 블록 크기 제한 확인
if totalSize+len(txBytes) > req.MaxTxBytes {
break
}
txs = append(txs, txBytes)
totalSize += len(txBytes)
iterator.Next()
}
return &abci.ResponsePrepareProposal{Txs: txs}, nil
}
```

8.2 ProcessProposal

```go
func (h *ProposalHandler) ProcessProposal(ctx sdk.Context, req *abci.RequestProc
// 1. 제안된 트랜잭션들 검증
for _, txBytes := range req.Txs {
```

```
tx, err := h.txDecoder(txBytes)
if  err != nil {
return &abci.ResponseProcessProposal{Status: abci.ResponseProcessProposal_REJECT
}
// 2. Ante Handler로 검증
_, err = h.anteHandler(ctx, tx, false)
if  err != nil {
return &abci.ResponseProcessProposal{Status: abci.ResponseProcessProposal_REJECT
}
}
return &abci.ResponseProcessProposal{Status: abci.ResponseProcessProposal_ACCEPT
}
```

9. EVM 실행

9.1 MsgEthereumTx 처리

```
func (k *Keeper) EthereumTx(goCtx context.Context, msg *types.MsgEthereumTx) (*t
ctx := sdk.UnwrapSDKContext(goCtx)
// 1. 트랜잭션 데이터 추출
txData, err := types.UnpackTxData(msg.Data)
if  err != nil {
return  nil, err
```

```
}

// 2. EVM 상태 머신 생성

evm := k.NewEVM(ctx, msg, txData)

// 3. 트랜잭션 실행

res, err := k.ApplyTransaction(ctx, evm, msg, txData)

if err != nil {

return nil, err

}

return res, nil

}
```

9.2 EVM 트랜잭션 실행

```
func (k *Keeper) ApplyTransaction(ctx sdk.Context, evm *vm.EVM, msg *types.MsgEt

// 1. 발신자 주소 추출

from, err := msg.GetSender(ctx.ChainID())

if err != nil {

return nil, err

}

// 2. 트랜잭션 타입별 처리

var result *vm.ExecutionResult

if txData.GetTo() == nil {

// 컨트랙트 배포
```

```
result, err = evm.Create(vm.AccountRef(from), txData.GetData(), txData.GetGas(),
} else {
// 컨트랙트 호출 또는 일반 전송
result, err = evm.Call(vm.AccountRef(from), *txData.GetTo(), txData.GetData(), t
}
if err != nil {
return nil, err
}
// 3. 실행 결과 처리
return k.processExecutionResult(ctx, result, from, txData)
}
```

10. Post Handler 처리

10.1 PostHandler 실행

```
func (ph PostHandler) PostHandle(ctx sdk.Context, tx sdk.Tx, simulate bool, suc
// 1. EVM 트랜잭션 필터링
for _, msg := range tx.GetMsgs() {
if msgEthTx, ok := msg.(*evmtypes.MsgEthereumTx); ok {
// 2. 트레이싱 데이터 수집
if ph.evmKeeper.IsTracingEnabled(ctx) {
traceData := ph.collectTraceData(ctx, msgEthTx)
```

```
ph.logger.Info("EVM transaction trace", "data", traceData)
}
// 3. 이벤트 발생
ctx.EventManager().EmitEvent(
sdk.NewEvent(
evmtypes.EventTypeEthereumTx,
sdk.NewAttribute(evmtypes.AttributeKeyTxHash, msgEthTx.Hash),
sdk.NewAttribute(evmtypes.AttributeKeyRecipient, msgEthTx.To()),
),
)
}
}
return  next(ctx, tx, simulate, success)
}
```

11. 인덱서 처리

11.1 ABCI 리스너

```
func (e *EVMIndexerImpl) ListenFinalizeBlock(ctx context.Context, req abci.Reque
if !e.enabled {
return  nil
}
```

```go
// 1. 인덱싱 작업 생성
task := &indexingTask{
req: &req,
res: &res,
args: e.extractIndexingArgs(ctx, req, res),
}
// 2. 인덱싱 채널에 전송 (비동기)
select {
case  e.indexingChan <- task:
return  nil
default:
e.logger.Warn("indexing channel full, dropping task")
return  nil
}
}
```

11.2 인덱싱 작업 처리

```go
func (e *EVMIndexerImpl) processIndexingTask(task *indexingTask) {
// 1. EVM 트랜잭션 추출
evmTxs := e.extractEVMTransactions(task.req, task.res)
// 2. 트랜잭션 데이터 인덱싱
```

```
for  i, evmTx := range  evmTxs {
// 트랜잭션 저장
e.TxMap.Set(ctx, evmTx.Hash.Bytes(), evmTx)
// 영수증 저장
receipt := e.generateReceipt(evmTx, task.res.TxResults[i])
e.TxReceiptMap.Set(ctx, evmTx.Hash.Bytes(), receipt)
// 해시 매핑 저장
cosmosTxHash := task.req.Txs[i]
e.TxHashToCosmosTxHash.Set(ctx, evmTx.Hash.Bytes(), cosmosTxHash)
e.CosmosTxHashToTxHash.Set(ctx, cosmosTxHash, evmTx.Hash.Bytes())
}
// 3. 블록 데이터 인덱싱
blockHeader := e.generateBlockHeader(task.req, task.res)
e.BlockHeaderMap.Set(ctx, uint64(task.req.Height), blockHeader)
// 4. 이벤트 발생
e.emitBlockEvents(blockHeader, evmTxs)
// 5. 인덱싱 높이 업데이트
e.lastIndexedHeight.Store(uint64(task.req.Height))
}
```

12. 에러 처리 및 복구

12.1 트랜잭션 실패 처리

```go
func (k *Keeper) handleTransactionError(ctx sdk.Context, err  error, gasUsed  ui

// 1. 가스 환불 계산

refund := k.calculateGasRefund(ctx, gasUsed)

// 2. 실패 영수증 생성

receipt := &types.Receipt{

Status: types.ReceiptStatusFailed,

CumulativeGasUsed: gasUsed,

GasUsed: gasUsed,

Logs: []*types.Log{},

}

// 3. 에러 이벤트 발생

ctx.EventManager().EmitEvent(

sdk.NewEvent(

types.EventTypeEthereumTxFailed,

sdk.NewAttribute(types.AttributeKeyError, err.Error()),

),

)

return &types.MsgEthereumTxResponse{

Hash: common.Hash{}.Hex(),

Logs: []*types.Log{},

Ret: []byte{},

VmError: err.Error(),
```

```
GasUsed: gasUsed,
}
}
```

12.2 상태 롤백

```
func (k *Keeper) executeWithRevert(ctx sdk.Context, fn func() error) error {
// 1. 상태 스냅샷 생성
snapshot := ctx.MultiStore().CacheMultiStore()
// 2. 함수 실행
err := fn()
if err != nil {
// 3. 에러 시 롤백
snapshot.Discard()
return err
}
// 4. 성공 시 커밋
snapshot.Write()
return nil
}
```

이러한 복잡한 트랜잭션 처리 플로우를 통해 MiniEVM은 Ethereum 호환성을 유지하면서도 Cosmos SDK의 모든 기능을 활용할 수 있는 통합된 환경을 제공합니다.